

Assignment 7 Report

Secure Network Application Development Using TCP

Name: Parth Rawat

Roll No.: 0126CY231042

Course: Assignment 7

Topic: Authentication, password hashing, session management, input validation, and Wireshark verification

Prepared from the provided assignment brief and the experimental screenshots supplied for this submission.

Project Details

Student Name	Parth Rawat
Roll Number	0126CY231042
Assignment	Assignment 7 - Secure Network Application Development Using TCP
Submitted Deliverable	Formatted report PDF with screenshots and verification evidence

1. Objective

The objective of this assignment is to strengthen the GUI-based multi-client TCP chat application from Assignment 6 by adding practical security controls. The enhanced system performs username and password authentication, stores passwords securely with SHA-256 hashes, prevents duplicate logins, validates user input, blocks repeated failed logins temporarily, supports logout and inactivity timeout, and records security-related events without exposing plaintext credentials. The application is also verified using Wireshark on TCP port 5000, as required in the assignment brief.

2. Security Features Implemented

Feature	Purpose	Verification
User authentication	Checks username and password before opening the chat window.	Login success and failure screenshots
SHA-256 password storage	Avoids storing plaintext passwords in the user database.	Code logic and secure login flow
Duplicate login prevention	Rejects a second session for the same username.	Duplicate login screenshot
Input validation	Rejects invalid usernames, empty passwords, oversized messages, and unsupported commands.	Invalid input and error screenshots
Failed login protection	Temporarily locks the account after repeated failures.	Lockout screenshot
Session management	Supports logout and inactivity timeout.	Timeout screenshots
Secure logging	Stores event details without password leakage.	Server/security log handling

3. System Architecture

The project uses a client-server architecture inside a Mininet topology. Multiple GUI clients connect to a TCP server on port 5000. The server authenticates users, tracks active sessions, stores hashed credentials, and manages message routing. After authentication, the client can send broadcast messages or private messages. Security events are written to log files, and Wireshark is used to inspect the TCP traffic.

System Architecture



Figure 1: System architecture showing authentication, session management, secure logging, and Wireshark verification.

4. Implementation

4.1 Credential Storage and Password Hashing

User credentials are stored in a CSV or JSON file. Before saving or comparing a password, the server applies SHA-256 hashing. This prevents plaintext passwords from being written to disk or displayed in logs. The login request is accepted only when the supplied hash matches the stored hash.

4.2 Authentication and Duplicate Login Prevention

The server checks whether the username exists and whether the same account is already active in another session. If the account is already logged in, the second connection is rejected immediately. This prevents concurrent access using the same identity.

4.3 Input Validation

The server rejects invalid usernames, empty passwords, oversized messages, and commands that are not supported by the protocol. This keeps the application stable and reduces the risk of malformed input causing unexpected behavior.

4.4 Failed Login Protection

After five consecutive failed login attempts, the account is temporarily locked for 60 seconds. This slows down brute-force attempts and is clearly visible in the lockout screenshot.

4.5 Session Management and Secure Logging

Each authenticated user session is monitored for inactivity. If the user remains idle beyond the timeout period, the session is closed automatically. Logout events and timeout events are recorded without saving passwords or other sensitive values.

5. Testing and Results

The application was tested in a Mininet lab environment using the required single,5 topology. Connectivity was verified with nodes, net, and pingall. The following screenshots demonstrate the expected behavior of the application.

5.1 Login and Access-Control Tests

Login and Access-Control Tests

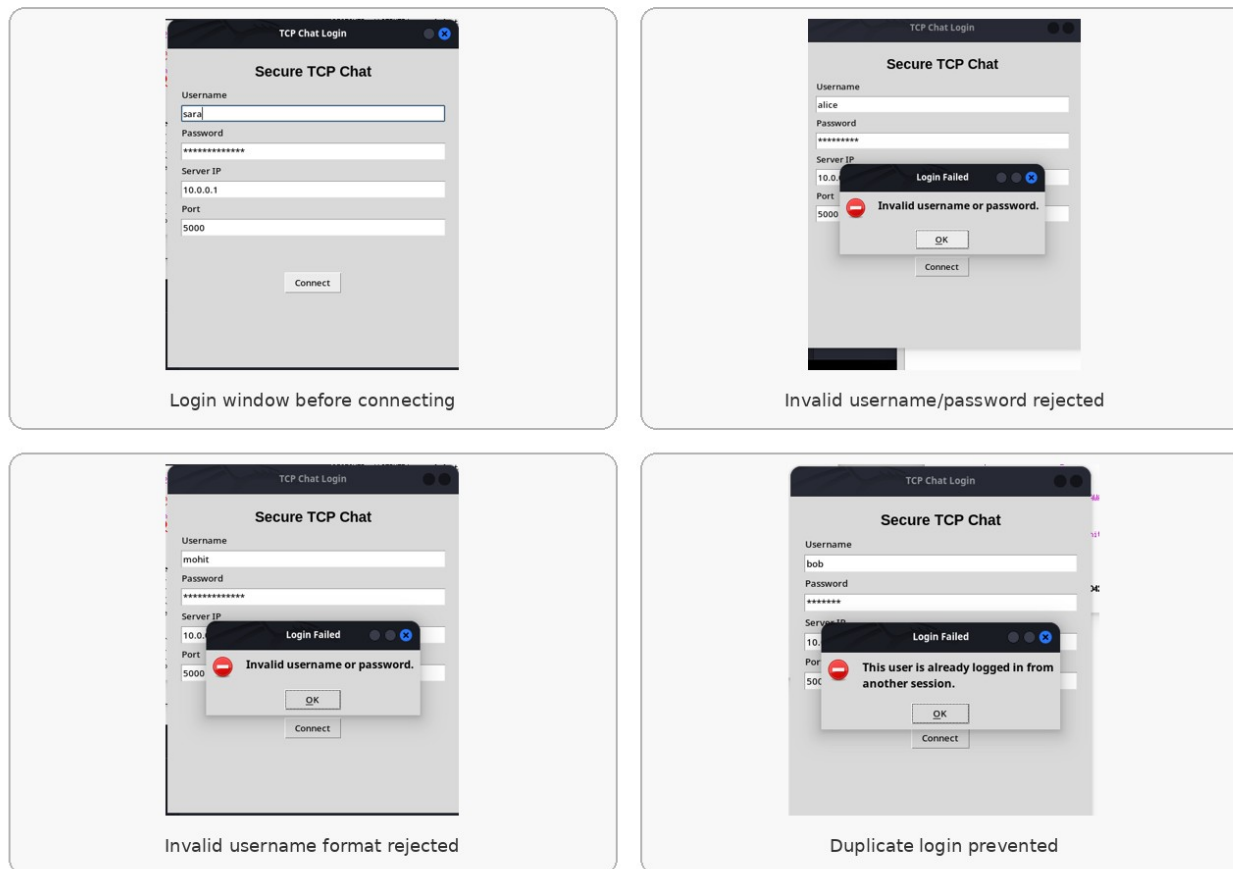


Figure 2: Login window, invalid credentials, invalid username format, and duplicate-login protection.

5.2 Security Enforcement Tests

Security Enforcement Tests

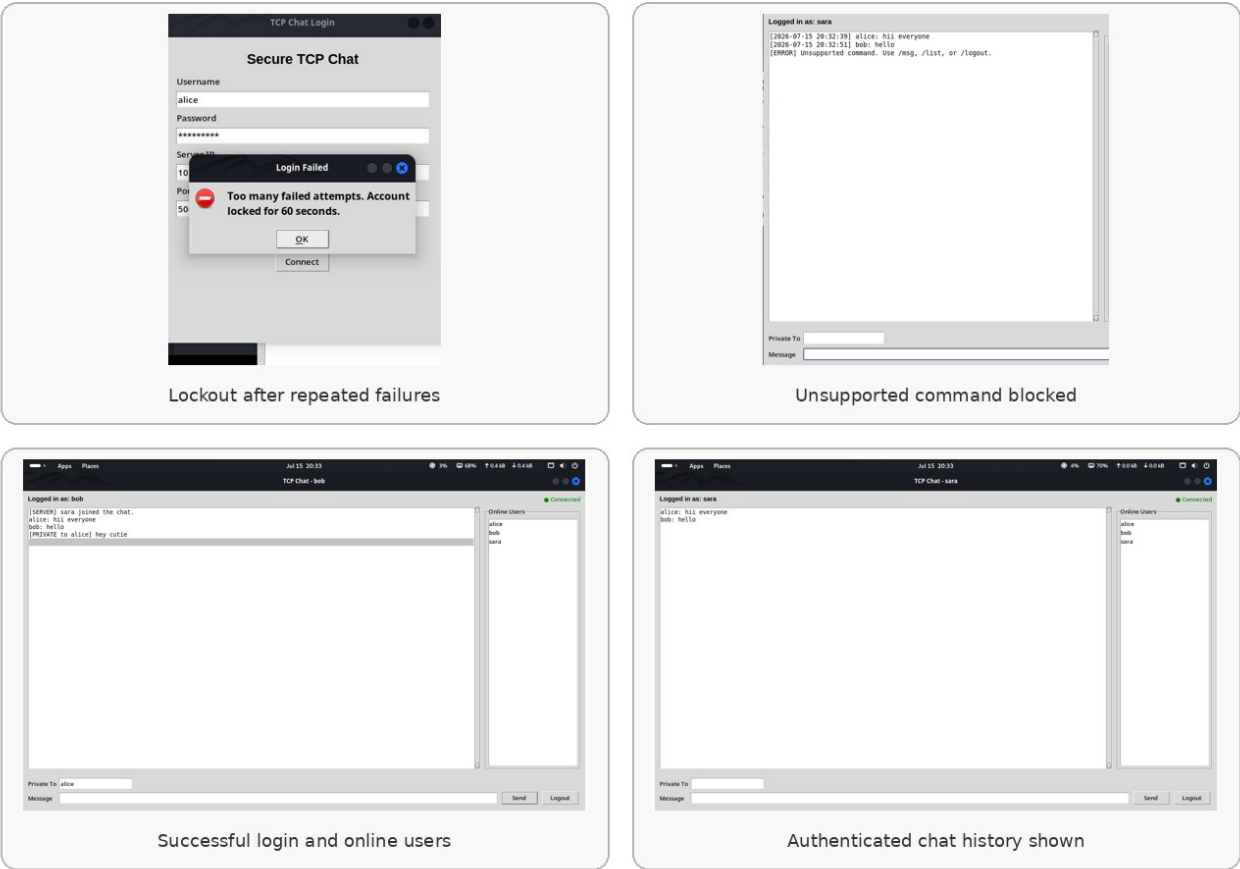


Figure 3: Lockout after repeated failures, unsupported command rejection, successful login, and authenticated chat history.

5.3 Session Management

Session Management

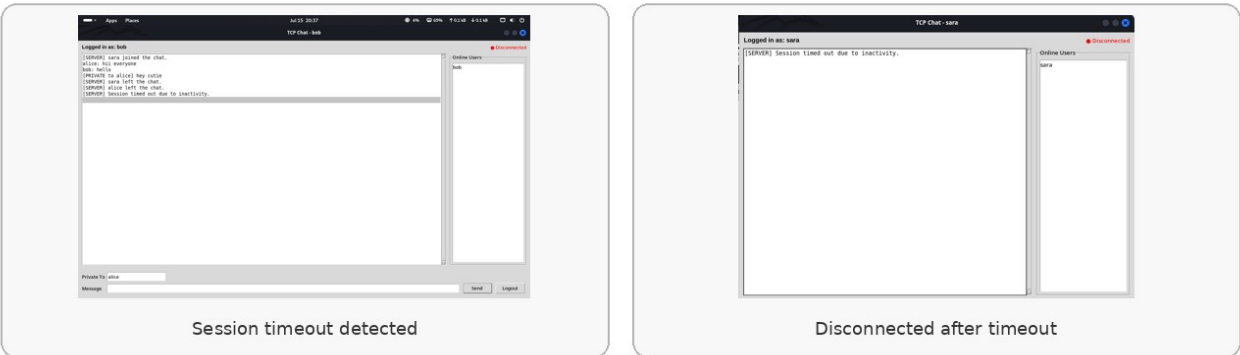


Figure 4: Inactivity timeout and automatic disconnection.

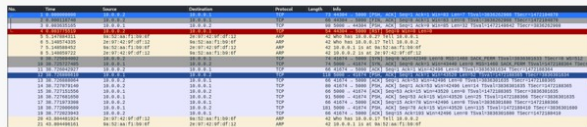
Test Case	Observed Result	Status
Valid login	Chat window opened and online users were displayed.	Pass

Invalid username/password	Login was rejected with an error dialog.	Pass
Duplicate login	Second session for the same username was blocked.	Pass
Too many failures	Account was locked temporarily after repeated failures.	Pass
Unsupported command	Server displayed an error and continued running.	Pass
Inactivity timeout	Client became disconnected after timeout.	Pass

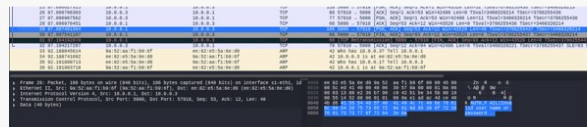
6. Wireshark Verification

Wireshark was used with the display filter `tcp.port == 5000` to observe the login exchange, failure cases, authenticated chat traffic, and logout behavior. The packet captures show the expected TCP conversation between client and server, which confirms that the application operates over the intended port and that security-related actions generate visible network traffic.

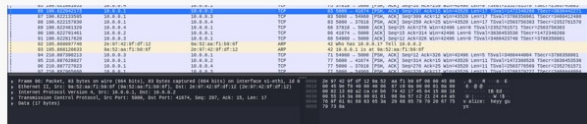
Wireshark Verification



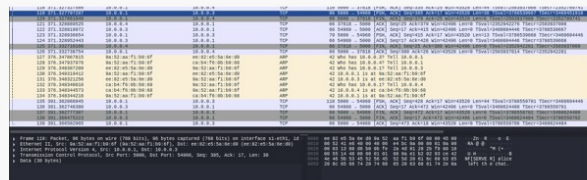
Wireshark: login capture



Wireshark: failed login



Wireshark: authenticated message



Wireshark: logout / disconnect

Figure 5: Wireshark captures for login, failed login, authenticated communication, and logout.

7. Conclusion

The Assignment 7 requirements were implemented as a practical security upgrade of the earlier GUI-based TCP chat application. The final system protects credentials with hashing, prevents duplicate logins, validates user input, limits repeated failed logins, supports session timeout and logout, and keeps security logging free of plaintext passwords. The provided screenshots and Wireshark captures demonstrate that the application behaves as required and that the TCP communication can be verified on port 5000. The handwritten reflection answers are intended to be submitted separately, as required by the assignment brief.